

Universidad Autónoma de San Luis Potosí

FACULTAD DE INGENIERÍA

CARRERA:

Ingeniería en Sistemas Inteligentes

ASIGNATURA:

Lenguajes de programación

ELABORADO POR:

369664 Antonio Jiménez José Emilio
388531 Alvarez Puente Alan Antonio

DOCENTE:

CICLO ESCOLAR:

2025 - 2026

San Luis Potosí, S.L.P.

20 de mayo de 2026

Índice

1. Estructuras de Datos y Objetos	3
Entidad	3
Atributos	4
CDiccionario	5
2. Funciones Entidad	7
Menu Entidades	7
Captura Entidad	7
Alta Entidad	7
Busca Entidad	7
Get Cabecera Entidades	8
Lee Entidad	8
Escribe Cabecera Entidades	8
Escribe Entidad	8
Inserta Entidad	8
Diagrama Insercción en Entidad	9
Consulta Entidades	9
Baja Entidad	9
Elimina Entidad	9
Diagrama Eliminación en Entidad	9
Reescribe Entidad	9
Modifica Entidad	10
3. Funciones Atributo	11
Menu Atributos	11
Captura Atributo	11
Alta Atributo	11
Buscar Atributo	11
Lee Atributo	11
Escribe Atributo	11
Inserta Atributo	12
Diagrama Insercción en Atributo	12
Consultar Atributo	12
Baja Atributo	12
Elimina Atributo	13
Diagrama Eliminación en Atributo	13
Reescribe Atributo	13
Modifica Atributo	13
Carga Atributos	13

4. Funciones Bloque	15
Menu Datos	15
Captura Bloque	15
Compara Bloques	15
Alta Bloque	15
Reescribe Bloque	15
Lee Bloque	15
Escribe Bloque	16
Inserta Bloque	16
Diagrama Insercción en Bloque	16
Consulta Bloques	16
Busca Bloque	17
Modifica Bloque	17
Baja Bloque	17
Diagrama Eliminación en Bloque	17
5. Diagrama General	18
Diagrama 1: Arquitectura del diccionario	18
6. Archivo	19
7. Clave primaria	20
8. Conclusiones	21
Referencias	22

1 Estructuras de Datos y Objetos

Descripción de la Estructura Entidad:

La estructura es el molde inicial de nuestro Diccionario en formato de base de datos de acceso aleatorio. Se establece un alias `cadena` para asegurar que cada registro tenga el mismo tamaño.

- **cadena nombre:** Identificador único de cada registro de entidad (máximo 30 caracteres).
- **long atr:** Puntero tipo `long` que conecta la entidad con su lista de atributos.
- **long data:** Dirección física en el archivo donde se almacenan los registros de datos reales.
- **long sig:** Enlace a la siguiente entidad, permitiendo una estructura de lista ligada.

```
1 #ifndef ENTIDAD_H_INCLUDED
2 #define ENTIDAD_H_INCLUDED
3 #include <iostream>
4 #include <cstdio>
5
6 typedef char cadena[30];
7 struct Entidad
8 {
9     cadena nombre;
10    long atr;
11    long data;
12    long sig;
13 };
14 #endif
```

Estructura 1: Estructura de Datos de Entidad

Descripción de la Estructura Atributo:

Esta estructura es el molde para las columnas de nuestro Diccionario. Al igual que en la entidad, se usa el alias `cadena` para mantener el tamaño fijo en el archivo binario y evitar que los registros se desfasen. En esta estructura observamos las siguientes declaraciones:

- **cadena nombre:** Identificador del atributo dentro de la entidad.
- **unsigned int tipo:** Código numérico que define el tipo de dato (ej. entero, flotante, cadena).
- **unsigned int tamano:** Longitud en bytes que ocupará el valor del atributo en el registro.
- **char iskp:** Bandera (flag) que indica si el atributo funciona como llave primaria (Primary Key).
- **long sig:** Puntero lógico (offset) que enlaza con el siguiente atributo de la entidad.
- **cadena descripcion:** Información adicional o etiqueta descriptiva del campo.
- **char null:** Indicador booleano que define si el campo permite valores nulos.

```
1 #ifndef ATRIBUTO_H
2 #define ATRIBUTO_H
3
4 typedef char cadena [30];
5
6 typedef struct{
7     cadena nombre;
8     unsigned int tipo;
9     unsigned int tamano;
10    char iskp;
11    long sig;
12    cadena descripcion;
13    char null;
14 }Atributo;
15
16 #endif
```

Estructura 2: Estructura de Datos de Atributos

Descripción del Objeto CDiccionario:

La clase CDiccionario funciona como el administrador principal del sistema. En su parte privada, contiene las variables necesarias para controlar el flujo del archivo binario y saber exactamente en qué parte del disco estamos parados mientras el usuario navega. En esta sección observamos las siguientes declaraciones:

- **FILE *arch:** Es el puntero al archivo físico donde se almacenan todas las entidades, atributos y datos.
- **Entidad activa:** Representa la entidad que el usuario está manipulando o consultando en ese momento.
- **long dir:** Guarda la dirección física (offset) actual dentro del archivo para las operaciones de lectura/escritura.
- **int nAtributos:** Es un contador que lleva el registro de cuántos atributos tiene la entidad seleccionada.
- **long tambloque:** Almacena el tamaño del bloque activo para gestionar correctamente el espacio en el archivo.
- **char nombreArchivo[50]:** Es el arreglo donde se guarda el nombre del archivo que el usuario decide abrir o crear.

```
1  #ifndef CDICCIONARIO_H_INCLUDED
2  #define CDICCIONARIO_H_INCLUDED
3
4  #include <cstdio>
5  #include "Atributo.h"
6  #include "Entidad.h"
7
8  class CDiccionario
9  {
10 private:
11     FILE *arch;
12     Entidad activa;
13     long dir;
14     int nAtributos;
15     long tambloque;
16
17     char nombreArchivo[50];
18 public:
19     //constructor
20     CDiccionario();
21
22     //Entidades
23     void Menu_Entidades();
24     Entidad CapturaEntidad();
25     void altaEntidad();
26     long buscaEntidad(char nombre[30]);
27     long getcabeceraEntidades();
```

```
28     Entidad leeEntidad(long dir);
29     void EscribeCabEntidades(long cab);
30     long escribeEntidad(Entidad ant);
31     void insertaEntidad(Entidad nvo, long dir);
32     void ConsultaEntidades();
33     void BajaEntidad();
34     long EliminaEntidad(char nombre[30]);
35     void reescribeEntidad(long dir, Entidad ant);
36     void modificaEntidad();
37
38     //Atributos
39     void Menu_Atributos();
40     void consultarAtributo();
41     long eliminaAtributo(char cad[30]);
42     void modificaAtributo();
43     void AltaAtributo();
44     Atributo CapturaAt();
45     void insertaAtributo(Atributo nvo, long dir);
46     long BuscarAtributo(char *atr);
47     Atributo leeAtributo(long dir);
48     long EscribeAtributo(Atributo ant);
49     void reescribeAtributo(long dir, Atributo atr);
50     void BajaAtributo();
51
52     //Datos
53     void Menu_Datos();
54
55     //Principal
56     void menu();
57     void Nuevo();
58     void abrir();
59
60 };
61
62 #endif
```

Estructura 3: Estructura de Datos CDiccionario

2 Funciones

Entidad

Menu Entidades

Prototipo: `void CDiccionario::Menu_Entidades();`

La función `menu_Entidades()` muestra un menú interactivo por consola para administrar las funciones relacionadas a Entidades. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar entidades, por consiguiente permitir la navegación hacia la edición de sus atributos y datos.

Captura Entidad

Prototipo: `Entidad CDiccionario::CapturaEntidad();`

La función `CapturaEntidad()` es la plantilla principal para crear registros, se usa una variable de tipo `Entidad`, primero se le pide al usuario el nombre de la entidad, después se inicializa las variables `nueva.atr` conexión con la lista de atributos", `nueva.data` "la dirección real de donde se encuentran los datos de los bloques, y `nueva.sig` que es la dirección a la siguiente entidad, con esto se crea un registro nuevo con sus listas vacías.

Alta Entidad

Prototipo: `void CDiccionario::altaEntidad();`

Esta función es la encargada de verificar que todo esté en condiciones para que se cree una entidad, primero manda a llamar `CapturaEntidad` para crear un registro temporal, seguido de esto viene la verificación con la función `buscaEntidad` para verificar que no haya una entidad con ese mismo nombre, en caso que no exista usa la función `escribeEntidad(nueva)` y guarda la dirección en un `long dir` para finalmente usar la función `insertaEntidad` para escribir en el archivo.

Busca Entidad

Prototipo: `long CDiccionario::buscaEntidad(char nombre[1]);`

La función `long CDiccionario::buscaEntidad(char nombre[])` verifica si se encuentra la entidad, en caso de que sí la encuentre dentro del recorrido `while(cab != -1)` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Get Cabecera Entidades

Prototipo: `long CDiccionario::getcabeceraEntidades();`

Esta función es la encargada de obtener la dirección dentro del archivo de donde inicia nuestra lista de entidades, empieza usando un `long dir`, para después con `fseek` dirigirse al inicio de nuestro archivo, con un `fread` extraemos la dirección y la guardamos en la variable `long dir`, sabemos que esa es la cabecera ya que todo el programa está construido para que en el inicio del archivo se encuentra nuestra cabecera de la lista de entidades.

Lee Entidad

Prototipo: `Entidad CDiccionario::leeEntidad(long dir);`

Se encarga de extraer la entidad del archivo, esta función llega con una dirección, esa dirección a partir del `buscaEntidad`, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Cabecera Entidades

Prototipo: `void CDiccionario::EscribeCabEntidades(long cab);`

La función empieza con un paso de parámetro de la cabecera, dentro de esta se mueve al inicio del archivo y en ese inicio escribe lo que haya tenido el parámetro de nombre `cab`.

Escribe Entidad

Prototipo: `long CDiccionario::escribeEntidad(Entidad ant);`

Esta función se encarga de registrar físicamente una nueva entidad en el archivo, donde se le pasa una copia de la entidad, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Entidad

Prototipo: `void CDiccionario::insertaEntidad(Entidad nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica una nueva entidad verificando tres casos: si el archivo está vacío, si va al inicio apoyándose de un `strcmp`, o si busca su lugar usando un ciclo `while`. En esencia, como `escribeEntidad` siempre guarda físicamente los datos al final, esta función solo reconecta las variables `nvo.sig` para que apunten de manera correcta y mantengan el orden.

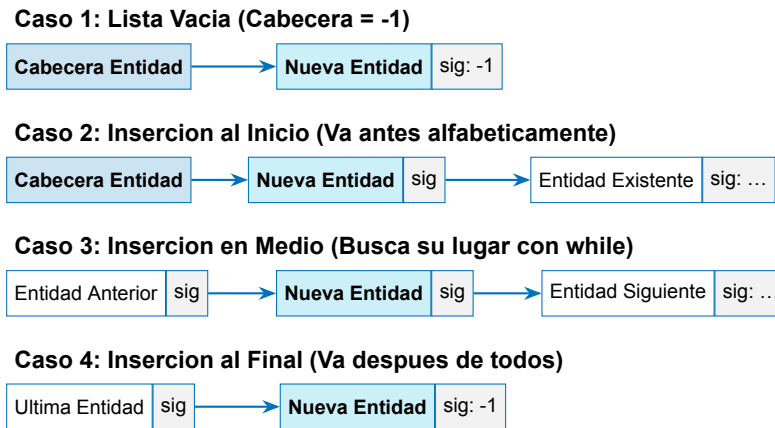


Diagrama 1: Representacion grafica de los 4 casos de insercion logica para la lista de Entidades.

Consulta Entidades

Prototipo: `void CDiccionario::ConsultaEntidades();`

Esta función es la encargada de imprimir en consola las entidades que se encuentran dentro de nuestro archivo. Para ello, hace uso de la función `getcabeceraEntidades` para ver dónde inicia la cabecera de la entidad; subsecuentemente hace uso de un ciclo `while (cab != -1)`, el cual va imprimiendo cada entidad con la dirección de sus componentes.

Baja Entidad

Prototipo: `void CDiccionario::BajaEntidad();` Su única función es preguntar al usuario por el nombre de la entidad, este nombre se lo pasa a la función `EliminaEntidad` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, muestra un aviso que fue eliminado.

Elimina Entidad

Prototipo: `long CDiccionario::EliminaEntidad(char nombre[]);` Esta función empieza haciendo una verificación, si el archivo está vacío regresa un `-1`, para después verificar si el lugar actual donde está parado es la entidad a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar donde está, uso una variable anterior para poder hacer la desconexión de la entidad, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeEntidad`.

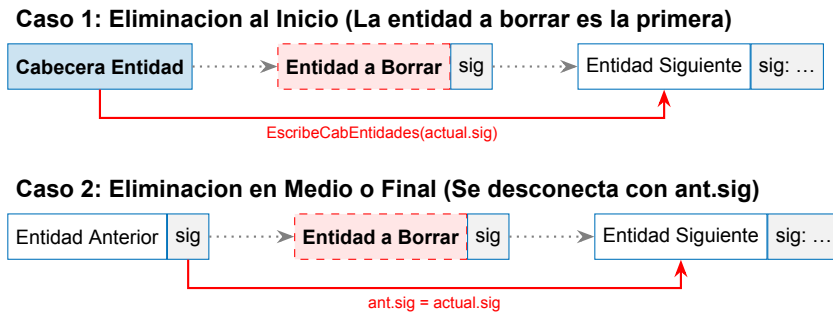


Diagrama 2: Representacion grafica de los 2 casos de baja logica para la lista de Entidades. El registro no se borra fisicamente, solo se puentea.

Reescribe Entidad

Prototipo: `void CDiccionario::reescribeEntidad(long dir, Entidad ant);`

Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Entidad

Prototipo: `void CDiccionario::modificaEntidad();`

Esta función permite actualizar los datos de un registro. Tras verificar que existencia y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `EliminaEntidad` y despues guarda y reubica la entidad actualizada utilizando `escribeEntidad` e `insertaEntidad`.

3 Funciones

Atributo

Menu Atributos

Prototipo: void CDiccionario::Menu_Atributos();

La función Menu_Atributos() muestra un menú interactivo por consola para administrar las funciones relacionadas a Atributos. Funciona por medio de un ciclo do-while y un switch-case que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar atributos, por consiguiente permitir la navegación hacia la edición de sus componentes.

Captura Atributo

Prototipo: Atributo CDiccionario::CapturaAt();

La función CapturaAt() es la plantilla principal para crear registros, se usa una variable de tipo Atributo, primero se le pide al usuario el nombre, después se pregunta el tipo y tamaño, y se inicializan las variables nuevoA.iskp y nuevoA.null, y nuevoA.sig en -1, con esto se crea un registro nuevo con sus datos listos.

Alta Atributo

Prototipo: void CDiccionario::AltaAtributo();

Esta función es la encargada de verificar que todo esté en condiciones para que se cree un atributo, primero manda a llamar CapturaAt para crear un registro temporal, seguido de esto viene la verificación con la función BuscarAtributo para verificar que no haya un atributo con ese mismo nombre, en caso que no exista usa la función EscribeAtributo(nuevoA) y guarda la dirección en un long dir para finalmente usar la función insertaAtributo para escribir en el archivo.

Buscar Atributo

Prototipo: long CDiccionario::BuscarAtributo(char *atr);

La función long CDiccionario::BuscarAtributo(char *atr) verifica si se encuentra el atributo, en caso de que sí lo encuentre dentro del recorrido while(cab != -1) regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, dentro del ciclo hace un string compare de la posición donde se encuentra y el nombre que se le haya pasado.

Lee Atributo

Prototipo: Atributo CDiccionario::leeAtributo(long dir);

Se encarga de extraer el atributo del archivo, esta función llega con una dirección, esa dirección a partir del BuscarAtributo, es usada para moverse dentro del archivo, hacer una copia y regresar esa copia para su posterior modificación o consulta.

Escribe Atributo

Prototipo: `long CDiccionario::EscribeAtributo(Atributo ant);`

Esta función se encarga de registrar físicamente un nuevo atributo en el archivo, donde se le pasa una copia del atributo, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo y en ese espacio de memoria usa un `fwrite` para guardarlo en el archivo.

Inserta Atributo

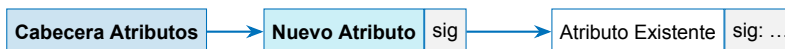
Prototipo: `void CDiccionario::insertaAtributo(Atributo nvo, long dir);`

Esta función se encarga de enlazar secuencialmente de manera lógica un nuevo atributo verificando tres casos: si en el caso que la cabecera `activa.atr` sea `-1` "Lista vacía", se enlaza directo, el segundo caso ocurre cuando el atributo se encuentra al inicio, o sea que el `strcmp` sea mayor a 0 decidiendo así que va antes, el tercer caso es cuando tiene que hacer una búsqueda apoyándose de un `while` que recorre toda la lista mientras no llegue al final, dentro usa otro `strcmp` para ver en qué lugar se encuentra su inmediato.
 newline En esencia la función hace reconexión entre la variable `nvo.sig` para que quede de manera ordenada, todos los `EscribeAtributo` siempre hacen el `write` al final por tanto lo único que cambia esta función son las direcciones a las cuales van apuntando de manera ordenada.

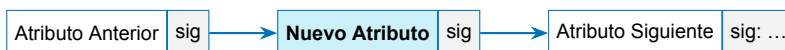
Caso 1: Lista Vacía (`activa.atr = -1`)



Caso 2: Insercion al Inicio (Va antes alfabeticamente)



Caso 3: Insercion en Medio (Busca su lugar con while)



Caso 4: Insercion al Final (Va despues de todos)



Diagrama 3: Representacion grafica de los 4 casos de insercion logica para la lista de Atributos.

Consultar Atributo

Prototipo: `void CDiccionario::consultarAtributo();`

Esta función es la encargada de imprimir en consola los atributos que se encuentran dentro de nuestro archivo. Para ello, hace uso de `activa.atr` para ver dónde inicia la cabecera del atributo; subsecuentemente hace uso de un ciclo `while` (`cab != -1`), el cual va imprimiendo cada atributo con la dirección de sus componentes.

Baja Atributo

Prototipo: `void CDiccionario::BajaAtributo();`

Su única función es preguntar al usuario por el nombre del atributo, este nombre se lo pasa a la función `eliminaAtributo` en caso de que no exista muestra en consola un aviso al usuario, por el contrario si existe, se hace la desconexión.

Elimina Atributo

Prototipo: `long CDiccionario::eliminaAtributo(char cad[]);`

Esta función empieza verificando la posición desde `activa.atr`, para después verificar si el lugar actual donde está parado es el atributo a borrar, en caso que no hace un recorrido donde hace uso de `while & strcmp`, para encontrar dónde está, usa una variable anterior para poder hacer la desconexión del atributo, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, lo actualiza en el archivo con la función `reescribeAtributo`.

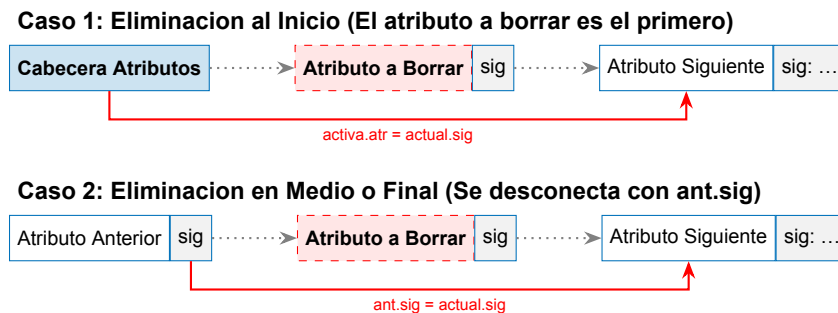


Diagrama 4: Representación gráfica de los 2 casos de baja lógica para la lista de Atributos.

Reescribe Atributo

Prototipo: `void CDiccionario::reescribeAtributo(long dir, Atributo atr);`

Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y una estructura con la información modificada, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir el bloque de datos. Es indispensable para actualizar los apuntadores lógicos durante operaciones de inserción y baja.

Modifica Atributo

Prototipo: `void CDiccionario::modificaAtributo();`

Esta función permite actualizar los datos de un registro. Tras verificar su existencia y capturar la nueva información, hace uso de una estrategia segura para mantener el orden alfabético de la lista: elimina lógicamente el registro antiguo con `eliminaAtributo` y después guarda y reubica el atributo actualizado utilizando `reescribeAtributo` e `insertaAtributo`.

Carga Atributos

Prototipo: `int CDiccionario::cargaAtributos();`

Esta función es la encargada de preparar todo para trabajar con los registros realizando tres tareas principales, primero recorre los atributos con un ciclo `while` para ir sumando su tamaño y definir el tamaño, después va guardando cada atributo dentro del arreglo `arrAtributos` e incrementa el contador `nAtributos`, y por último verifica si el atributo es clave primaria comprobando la variable `iskp` para intercambiarlo y ponerlo siempre en la posición 0 del arreglo, finalizando con el retorno del número de claves encontradas.

4 Funciones

Bloque

Menu Datos

Prototipo: `void CDiccionario::Menu_Datos();` La función `Menu_Datos()` muestra un menú interactivo por consola para administrar las funciones relacionadas a los Bloques o Registros. Funciona por medio de un ciclo `do-while` y un `switch-case` que lee la opción del usuario, para ejecutar acciones como dar de alta, consultar, eliminar o modificar registros, permitiendo la navegación hacia la edición de los datos.

Captura Bloque

Prototipo: `void* CDiccionario::CapturaBloque();`
La función `CapturaBloque()` es la plantilla principal para crear registros de datos, se usa un puntero `void* bloque` usando `malloc` con el tamaño `tambloque`, primero inicializa el apuntador al siguiente, después usa un ciclo `for` para pedirle al usuario los datos dependiendo del tipo de atributo, con esto se crea un bloque nuevo con su información lista.

Compara Bloques

Prototipo: `float CDiccionario::comparaBloques(void* b1, void *b2);`
Esta función es la encargada de comparar dos bloques apoyándose de la clave primaria que siempre está en el índice 0 del arreglo, primero verifica el tipo de dato, en el caso de ser una cadena hace uso de `strcmp` para compararlos, mientras que para los tipos numéricos enteros o flotantes realiza una resta directa de sus valores, regresando un resultado que sirve para saber si son iguales o decidir cuál va antes.

Alta Bloque

Prototipo: `void CDiccionario::altaBloque();`
Esta función es la encargada de verificar que todo esté en condiciones para crear un bloque, primero manda a llamar `CapturaBloque` para crear el registro, seguido de esto viene la verificación con la función `buscaBloque` para comprobar que no exista ya esa clave primaria, en caso que no exista usa `escribeBloque` obteniendo su dirección, y finalmente usa `InsertaBloque` para escribir en el archivo.

Reescribe Bloque

Prototipo: `void CDiccionario::reescribeBloque(void* bloque, long dir);`
Esta función se encarga de actualizar los datos de un registro ya existente en el archivo binario. Recibe una dirección y un bloque con la información, usa `fseek` con `SEEK_SET` para posicionarse exactamente en la dirección proporcionada, y `fwrite` para sobrescribir haciendo uso de la variable `tambloque`. Es indispensable para actualizar apuntadores.

Lee Bloque

Prototipo: `void* CDiccionario::LeeBloque(long dir);`

Se encarga de extraer el bloque del archivo, esta función llega con una dirección, la cual es usada por un `fseek` para moverse dentro del archivo y con un `fread` hacer una copia de tamaño `tambloque` en un espacio de memoria, regresando esa copia para su posterior modificación o consulta.

Escribe Bloque

Prototipo: `long CDiccionario::escribeBloque(void* bloque);`

Esta función se encarga de registrar físicamente un nuevo bloque en el archivo, donde se le pasa el puntero de los datos, la función usa un `fseek` con `SEEK_END` para ir hasta el final del último registro del archivo, guarda su dirección con `ftell` y en ese espacio usa un `fwrite` para guardarlo en el archivo, regresando la dirección física.

Inserta Bloque

Prototipo: `void CDiccionario::InsertaBloque(void *nvo, long dirnvo);`

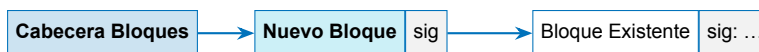
Esta función se encarga de enlazar secuencialmente de manera lógica un nuevo bloque, si en el caso que la cabecera `activa.data` sea `-1` se enlaza directo, si va al inicio o en medio usa `comparaBloques` dentro de un `while` para hacer una búsqueda y encontrar su lugar exacto.

newline En esencia la función hace reconexión de las direcciones para que quede de manera ordenada, como `escribeBloque` siempre guarda al final, lo único que cambia esta función son los apuntadores lógicos.

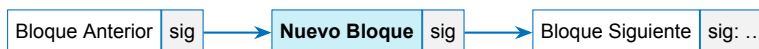
Caso 1: Lista Vacía (`activa.data = -1`)



Caso 2: Insercion al Inicio (`comparaBloques < 0`)



Caso 3: Insercion en Medio (Busca su lugar segun Clave Primaria)



Caso 4: Insercion al Final (Va despues de todos)



Diagrama 5: Representacion grafica de los 4 casos de insercion logica para la lista de Bloques de Datos.

Consulta Bloques

Prototipo: void CDiccionario::consultaBloques();

Esta función es la encargada de imprimir en consola los registros que se encuentran dentro del archivo. Para ello, hace uso de `activa.data` para ver dónde inicia la cabecera, imprime los nombres de los atributos, y subsecuentemente hace uso de un ciclo `while (cab != -1)` el cual va imprimiendo cada bloque haciendo las conversiones de tipo correspondientes.

Busca Bloque

Prototipo: long CDiccionario::buscaBloque(void *bloquebus);

La función `buscaBloque` verifica si se encuentra el registro, en caso de que sí lo encuentre dentro del recorrido `while(cab != -1)` comparando con la función `comparaBloques` regresa la dirección en donde se encuentra, en caso contrario regresa un valor de -1, moviéndose siempre leyendo el apuntador siguiente de cada bloque.

Modifica Bloque

Prototipo: void CDiccionario::modificaBloque();

Esta función permite actualizar los datos de un bloque. Empieza pidiendo los nuevos datos con `CapturaBloque`, luego hace un recorrido para encontrar el bloque actual y desconectarlo enlazando el anterior con el siguiente usando `reescribeBloque`, y después guarda y reubica el nuevo bloque actualizado utilizando `escribeBloque` e `InsertaBloque`.

Baja Bloque

Prototipo: void CDiccionario::bajaBloque();

Esta función empieza llamando a `CapturaBloque` para saber qué clave buscar, para después hacer un recorrido donde hace uso de `while` y `comparaBloques` para encontrar dónde está, usa una variable anterior para poder hacer la desconexión del bloque, no borra nada dentro del archivo solo desconecta y conecta con su inmediato siguiente, actualizando el archivo con `reescribeBloque`.

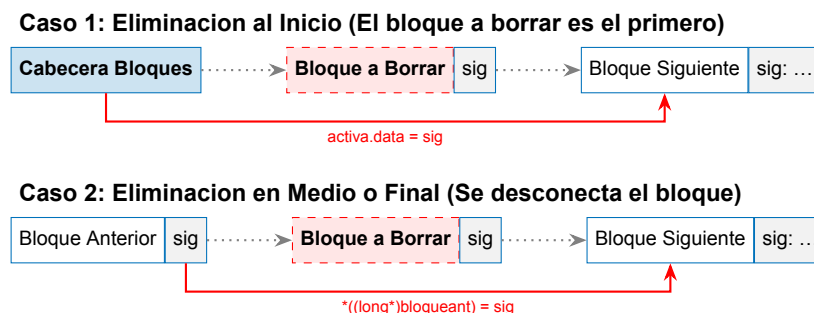


Diagrama 6: Representacion grafica de los 2 casos de baja logica para la lista de Bloques.

5 Diagrama General

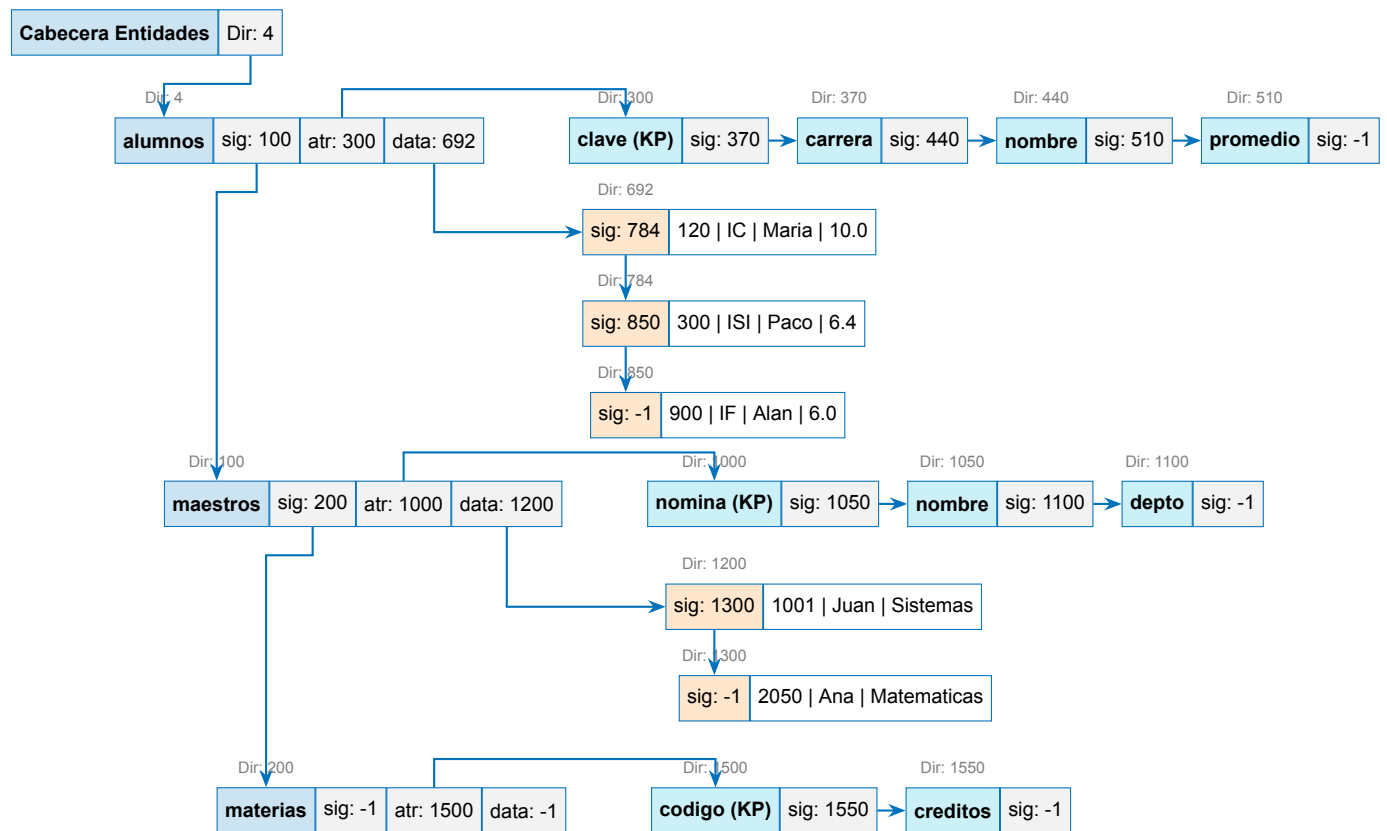


Diagrama 7: Diagrama 1: Arquitectura del diccionario mostrando la lista principal de Entidades y cómo cada registro conecta con sus respectivas listas de Atributos y Bloques de datos. El mapa ilustra la navegación física entre direcciones y punteros.

6 Archivo

7 Clave primaria

8 Conclusiones

Referencias
